



From strategic design to Modular, Eventful, and Reliable Systems

18.06.2025 • Mihaita Tinta - SWAT



do your thing

Agenda

- Strategic design
- Design tools
- Event based communication
- Live demo
- Implementation details
- Conclusions



What is Strategic Design?



do your thing

Split Mate Corporation

A platform to help friends split various costs (e.g., dinner, car rental, house expenses) using OpenBanking APIs for bank transfers.



Impact mapping

Goal: Help friends split various costs efficiently and securely.

Who: Customers

How - Impacts:

- Efficient Cost Splitting
- Secure Transactions
- User Experience



What - Deliverables:

- Cost Splitting Feature
- Transaction Security
- User Interface

Impact mapping

Goal: Help friends split various costs efficiently and securely.

Who: TPP/Banks

How - Impacts:

- Seamless Integration
- Increased Transactions



What - Deliverables:

- API Integration
- Compliance

Impact mapping

Goal: Help friends split various costs efficiently and securely.

Who: Split Mate Corporation

How - Impacts:

- Enter Romanian Market
- User Retention
- Revenue Growth



What - Deliverables:

- Marketing Campaign
- Customer Support
- Performance Testing
- Transaction Fees

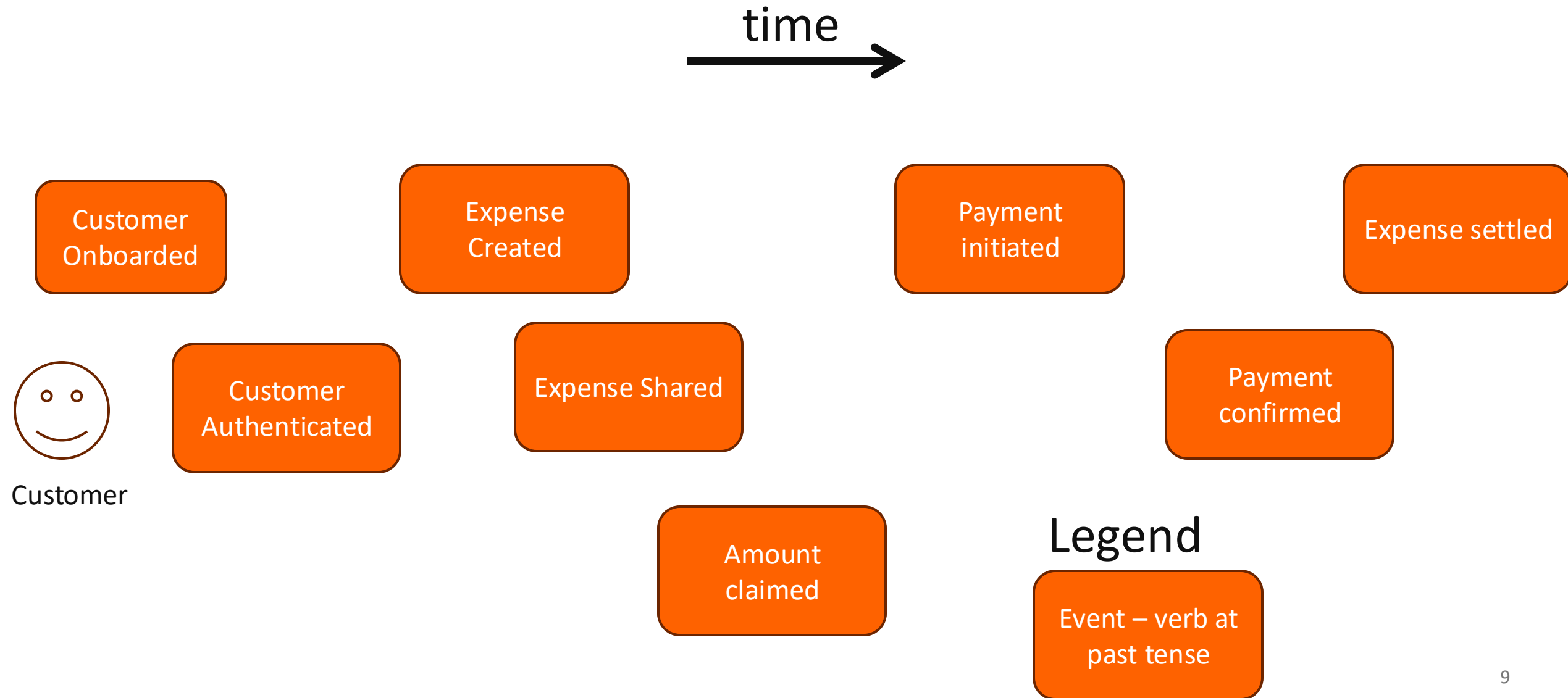


What is Event Storming?



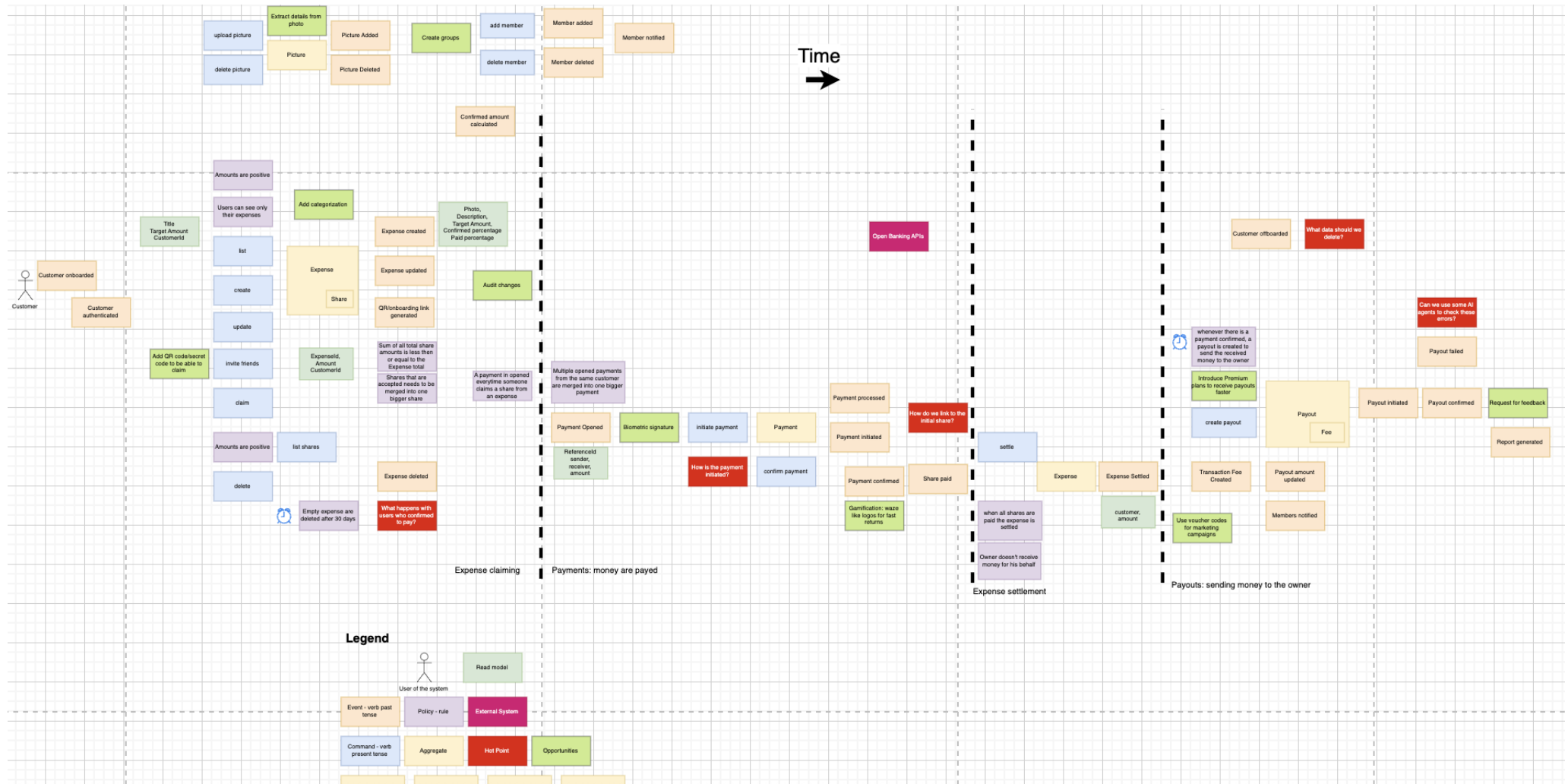
do your thing

Big Picture Event Storming



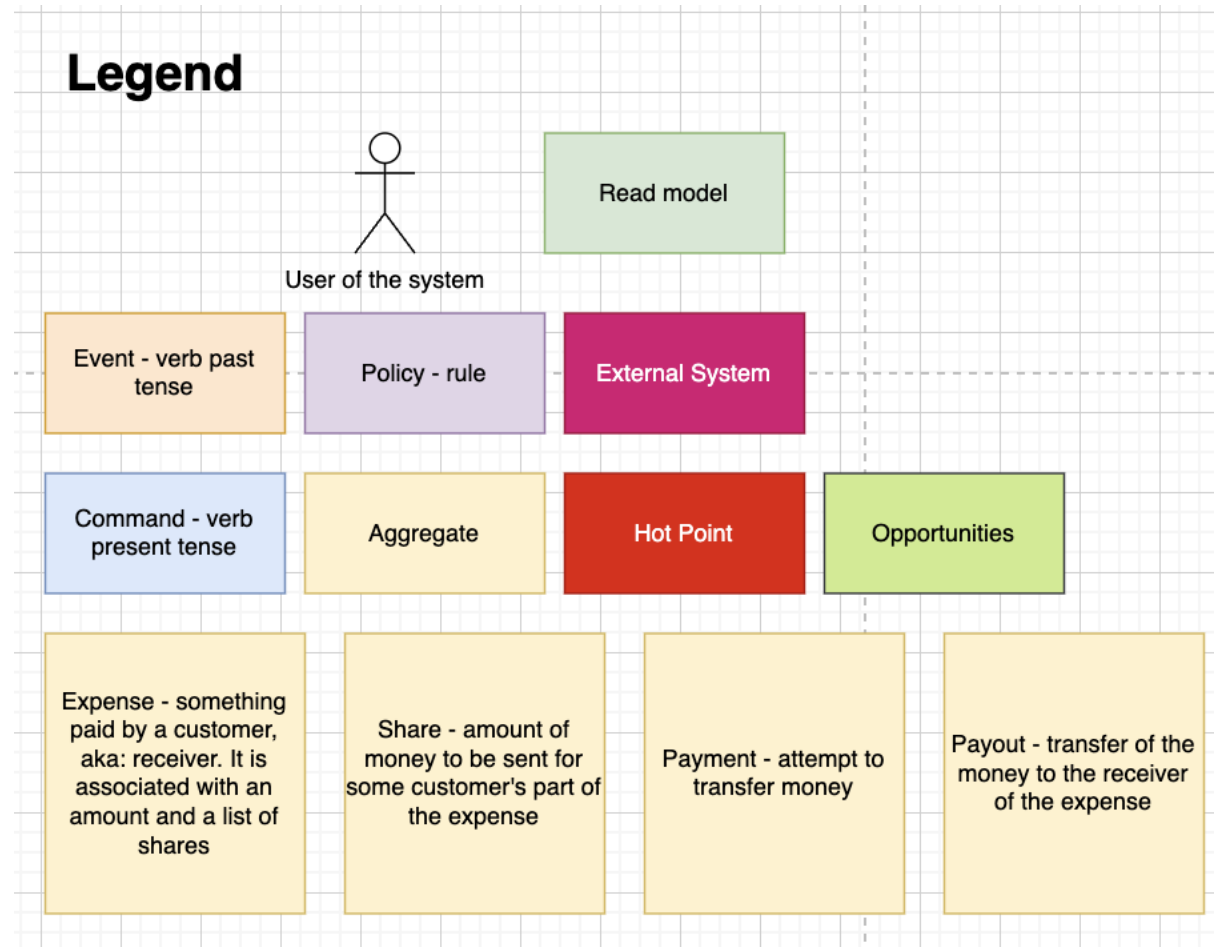
Event Storming

Cost splitting feature



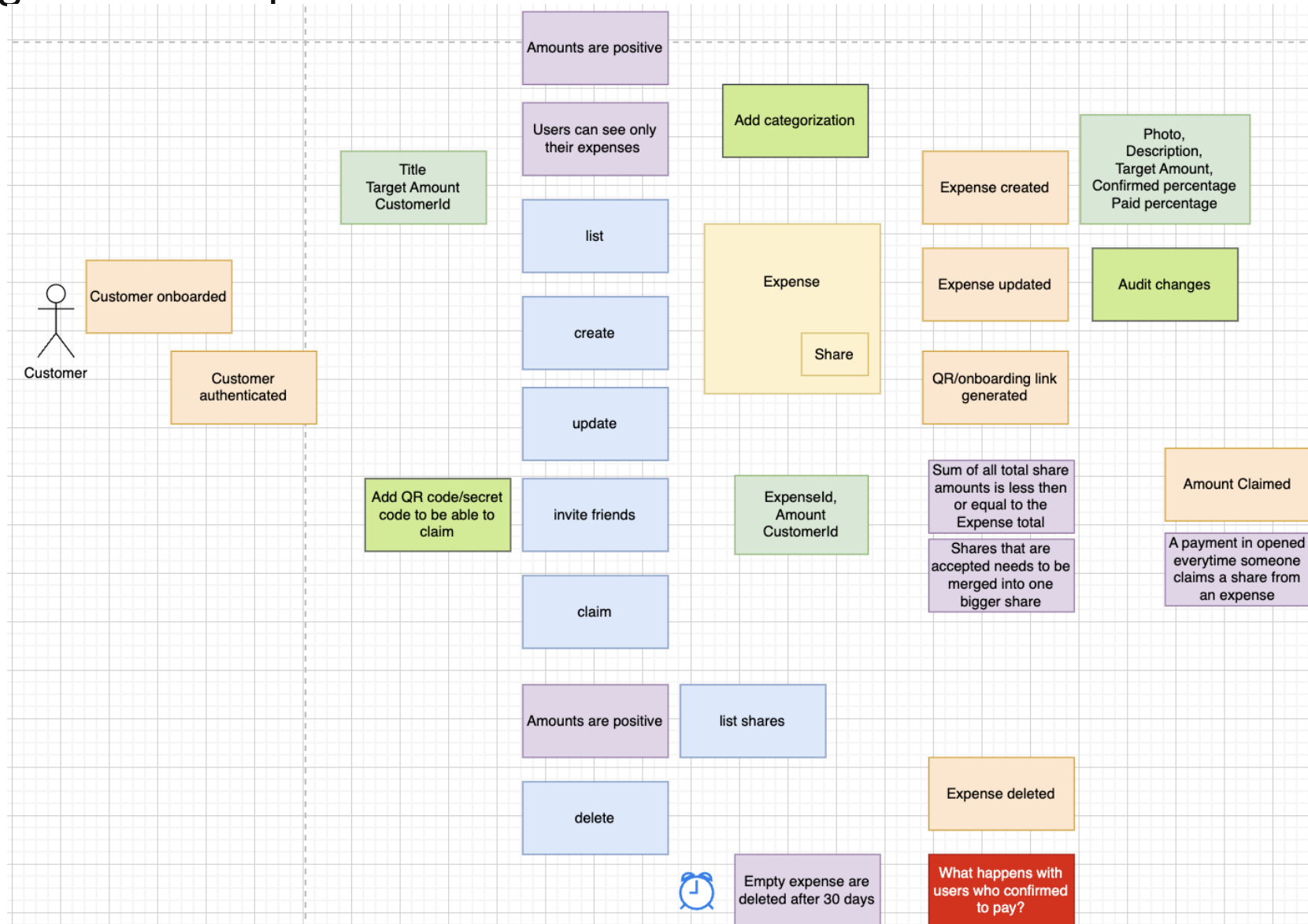
Event Storming

Cost splitting feature



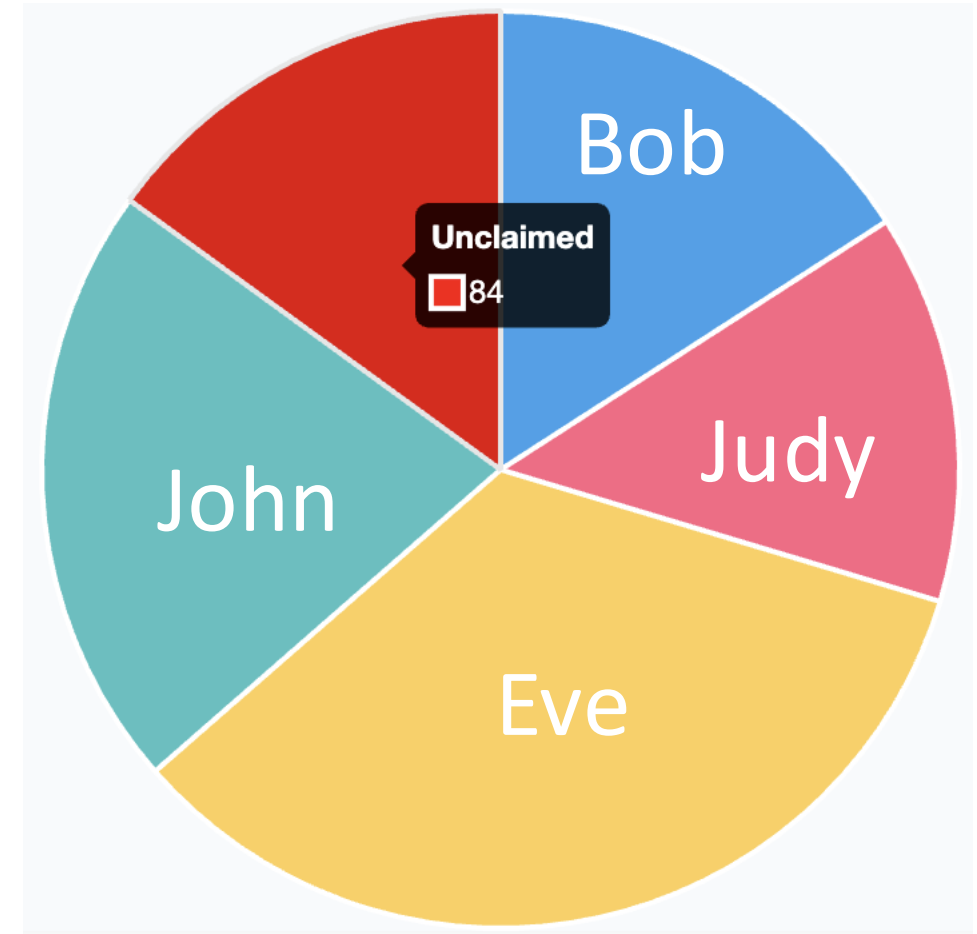
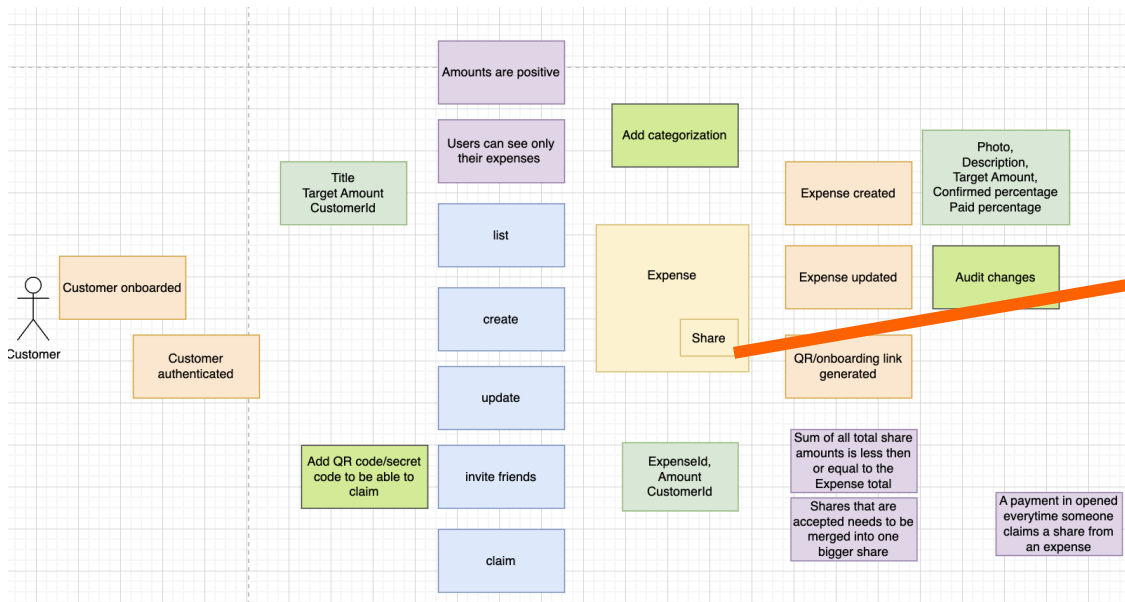
Event Storming

Cost splitting feature - Expenses



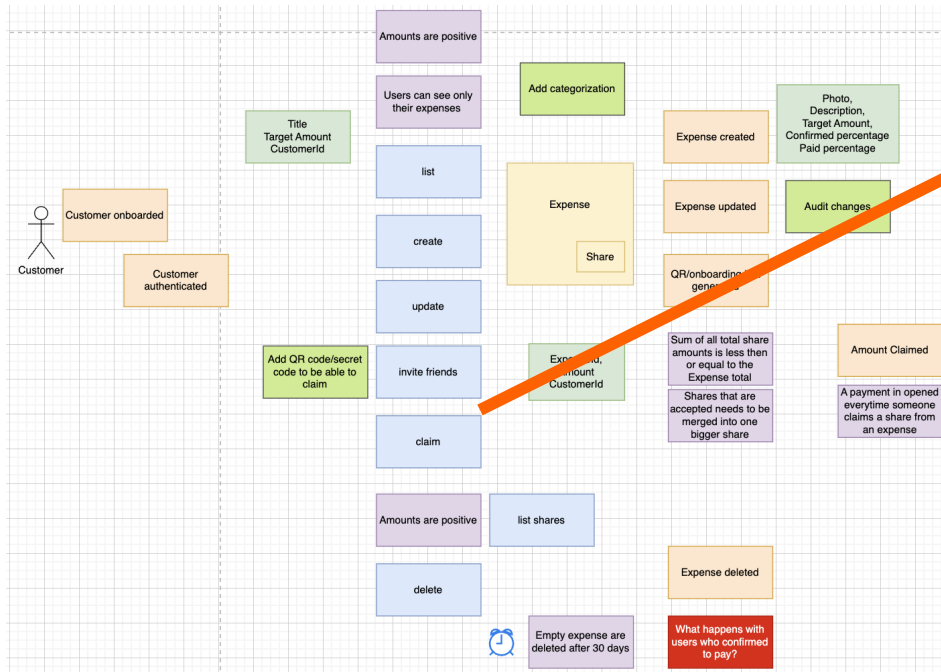
Event Storming

Cost splitting feature – Expenses implementation



Event Storming

Cost splitting feature – Expenses actions

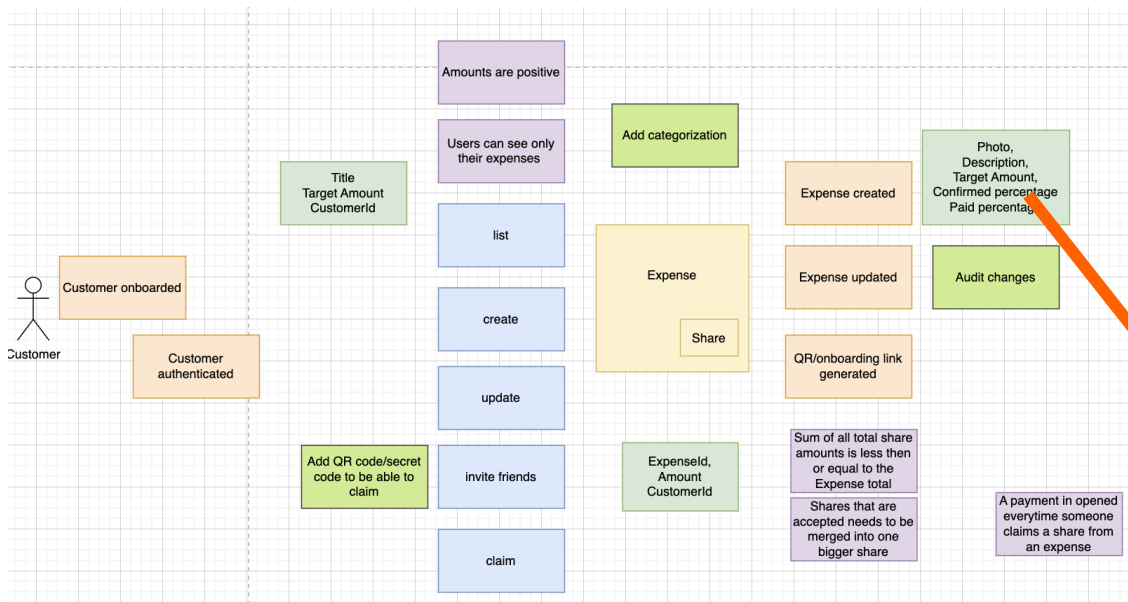


ExpenseManagement

```
list(CustomerIdentifier): List<Expense>
create(CreateExpenseRequest, CustomerIdentifier): Expense
expense(CustomerIdentifier, ExpenseIdentifier): Optional<Expense>
expenseByShareCode(ExpenseIdentifier, String): Optional<Expense>
inviteFriends(CustomerIdentifier, ExpenseIdentifier): QrCode
claimShare(ExpenseIdentifier, ClaimShareRequest, CustomerIdentifier): QrCode
```

Event Storming

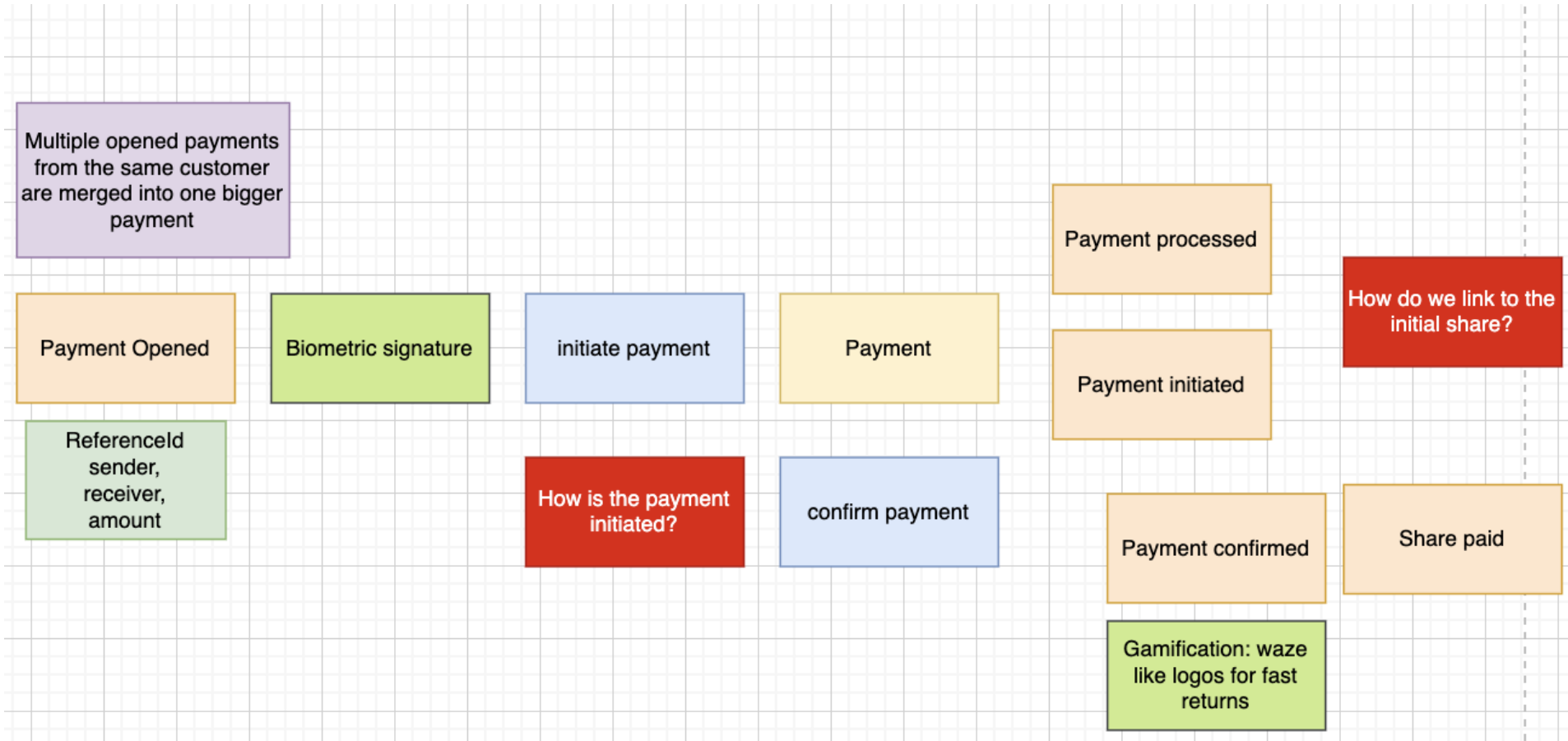
Cost splitting feature – Expenses implementation



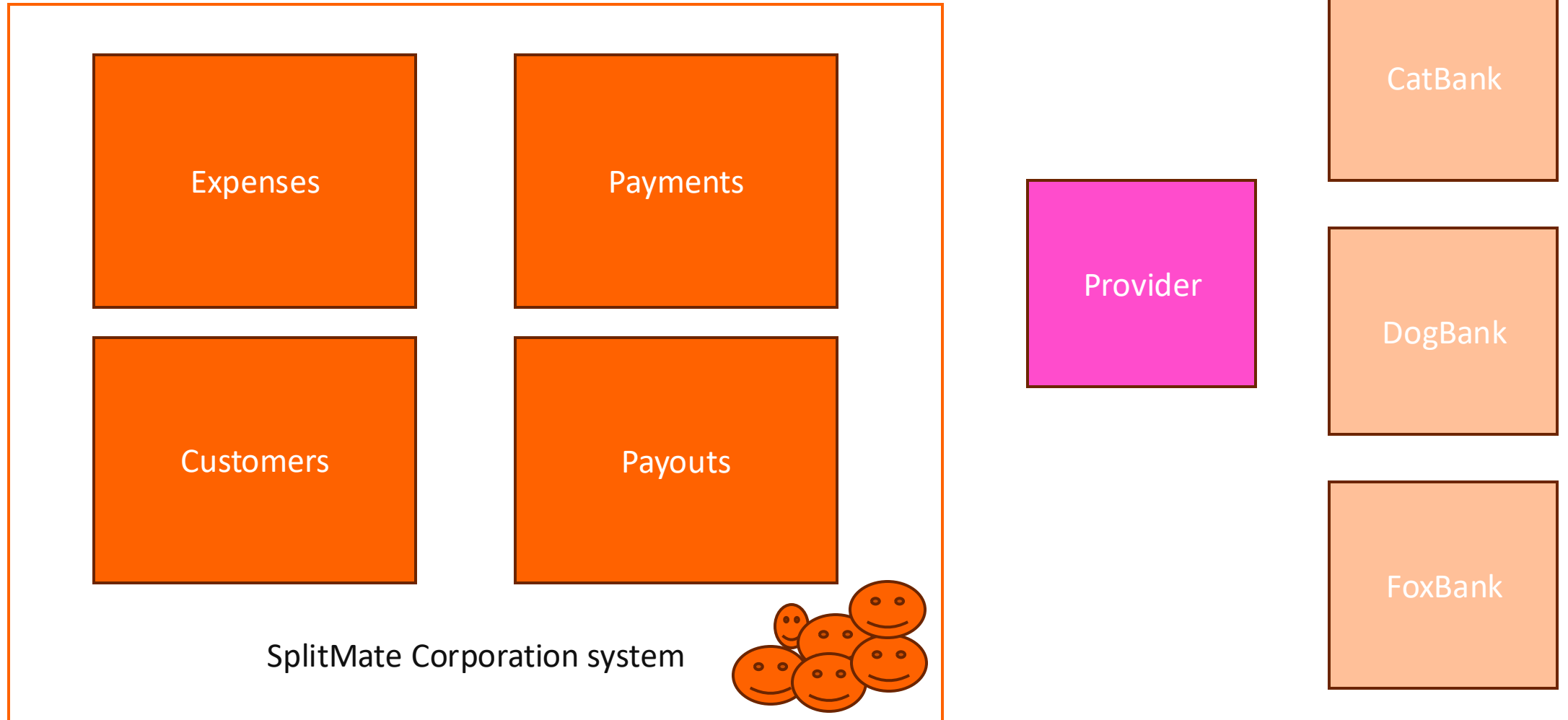
```
✓ Expense
  (m) claim(TargetAmount, CustomerIdentifier): Expense
  (m) shareToOthers(CustomerIdentifier): Expense
  (m) getShareNotPaidByUser(CustomerIdentifier): Optional<Share>
  (m) onPaymentConfirmed(PaymentConfirmed): Expense
  (m) settle(): boolean
  (m) isSettled(): boolean
  (m) getAmountClaimedButNotPaidByUser(CustomerIdentifier): BigDecimal
  (m) getRemainingToBePaid(): BigDecimal
  (m) getOwnedMoneyToReceiver(): BigDecimal
  (f) id: ExpenseIdentifier
  (f) userId: CustomerIdentifier
  (f) title: Title
  (f) shareCode: Optional<ShareCode>
  (f) targetAmount: TargetAmount
  (f) creationDate: CreationTime
  (f) shares: List<Share>
```

Event Storming

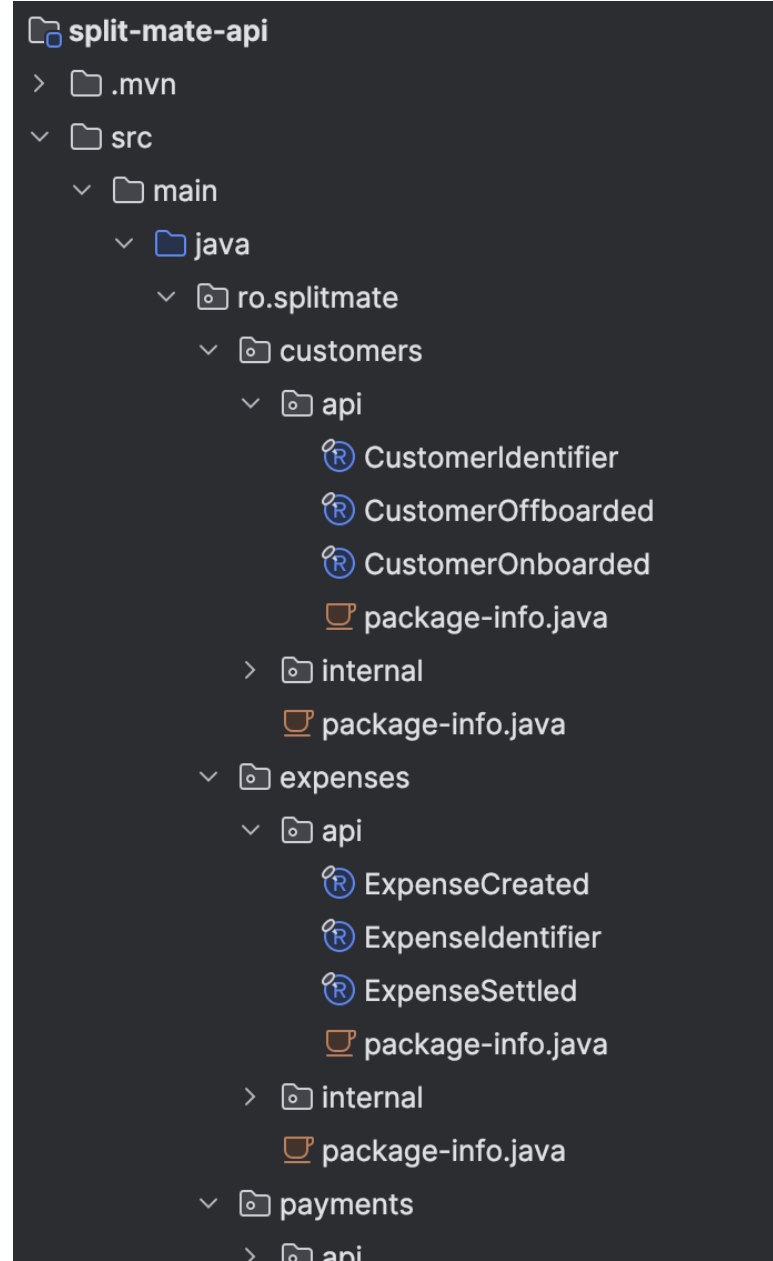
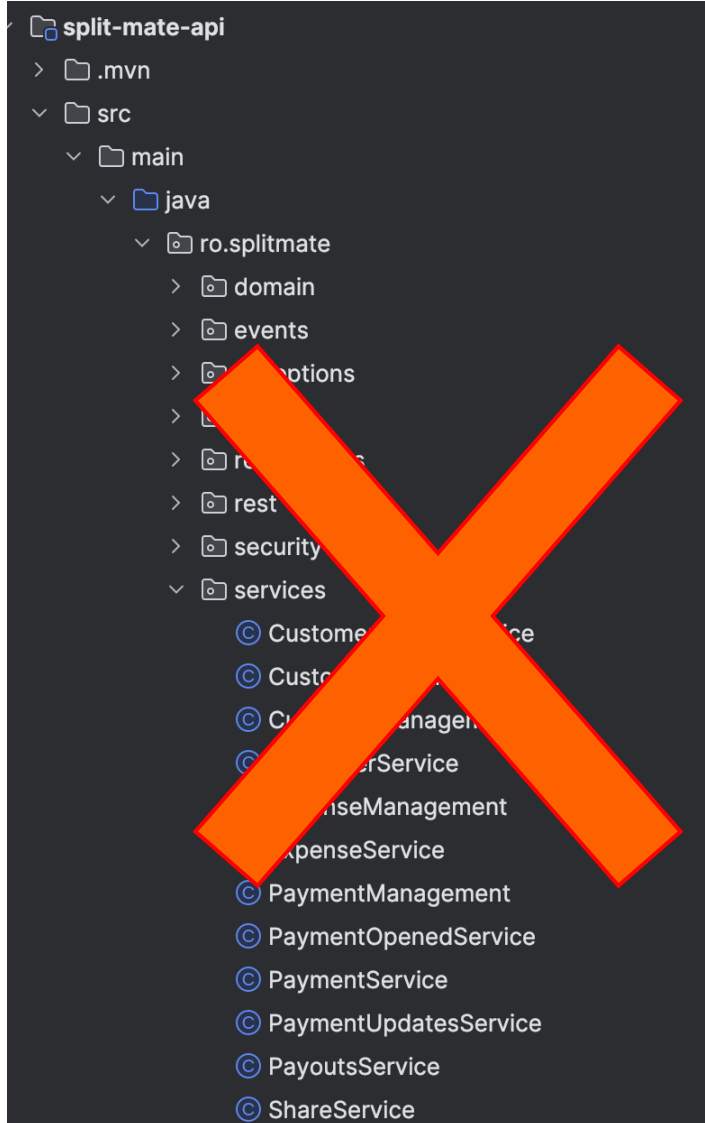
Cost splitting feature - Payments



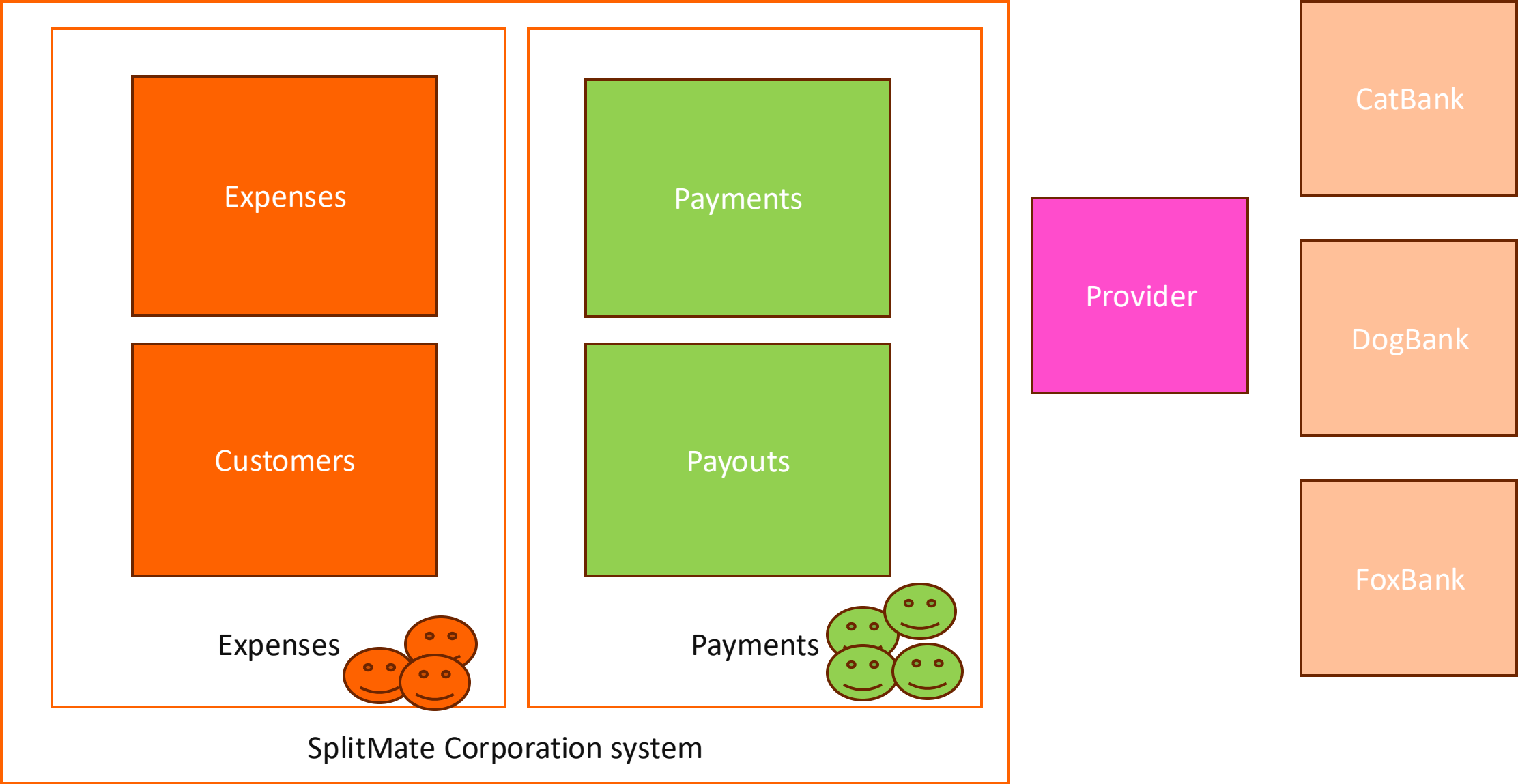
Application overview – 1st step



Application overview – 1st step

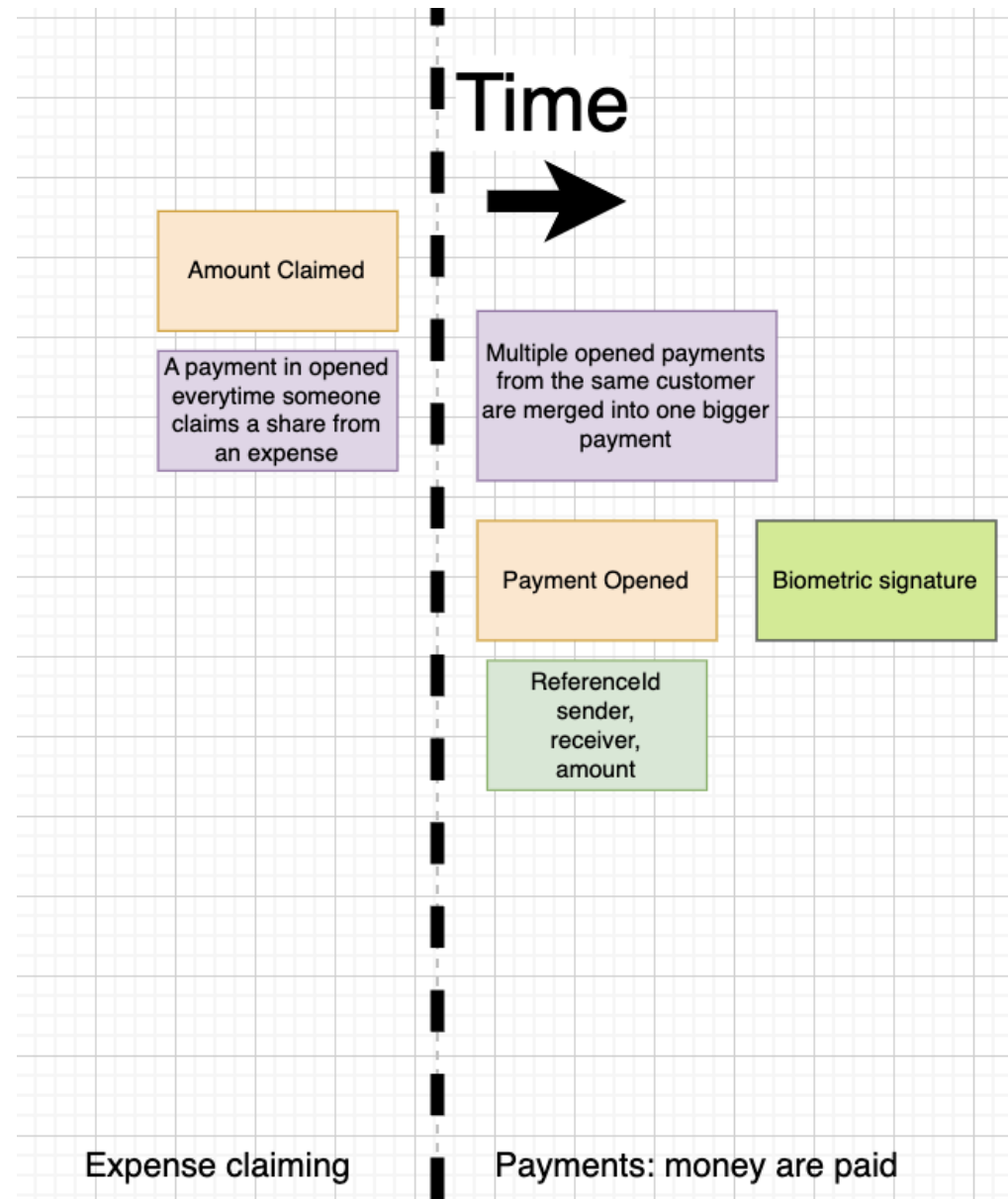


Application overview – 2nd step



Event Storming

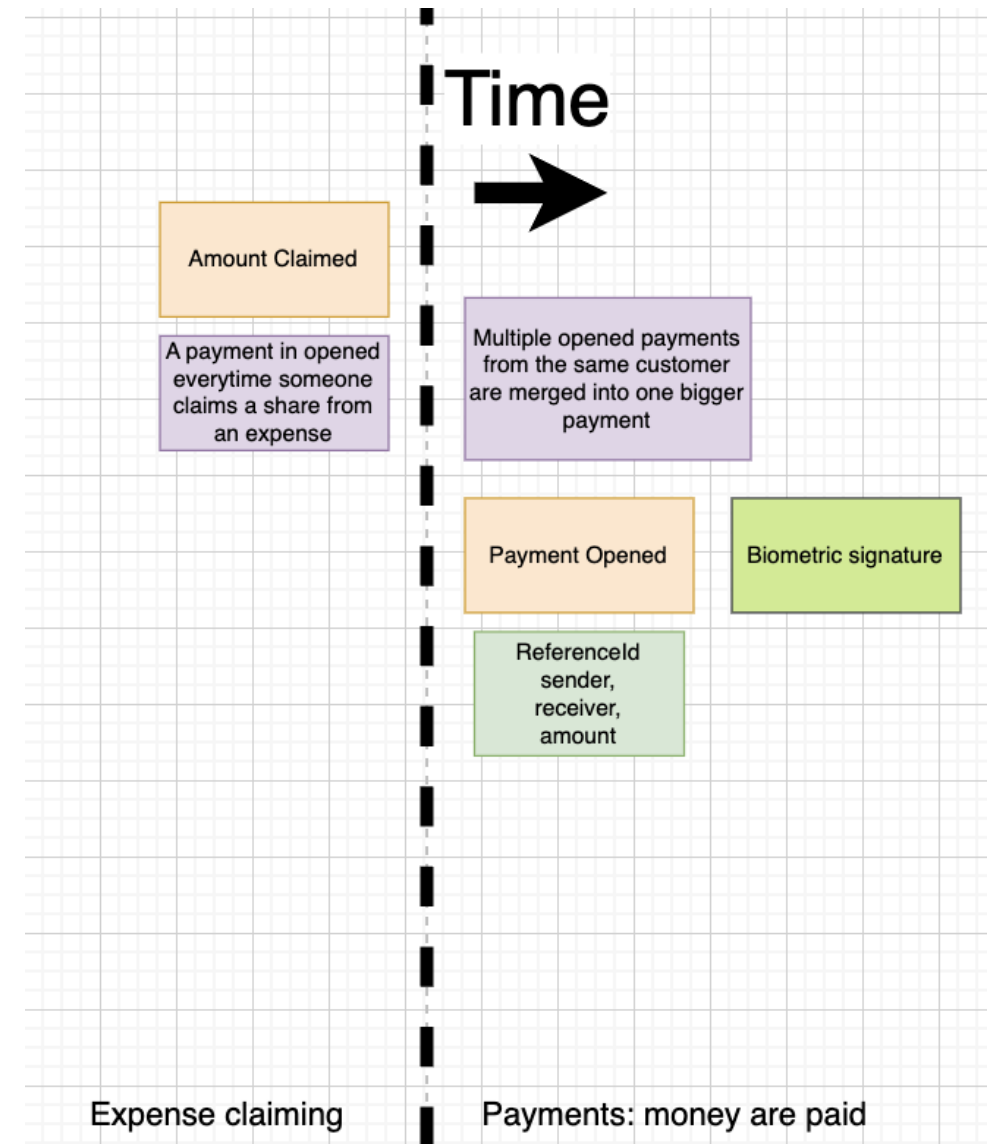
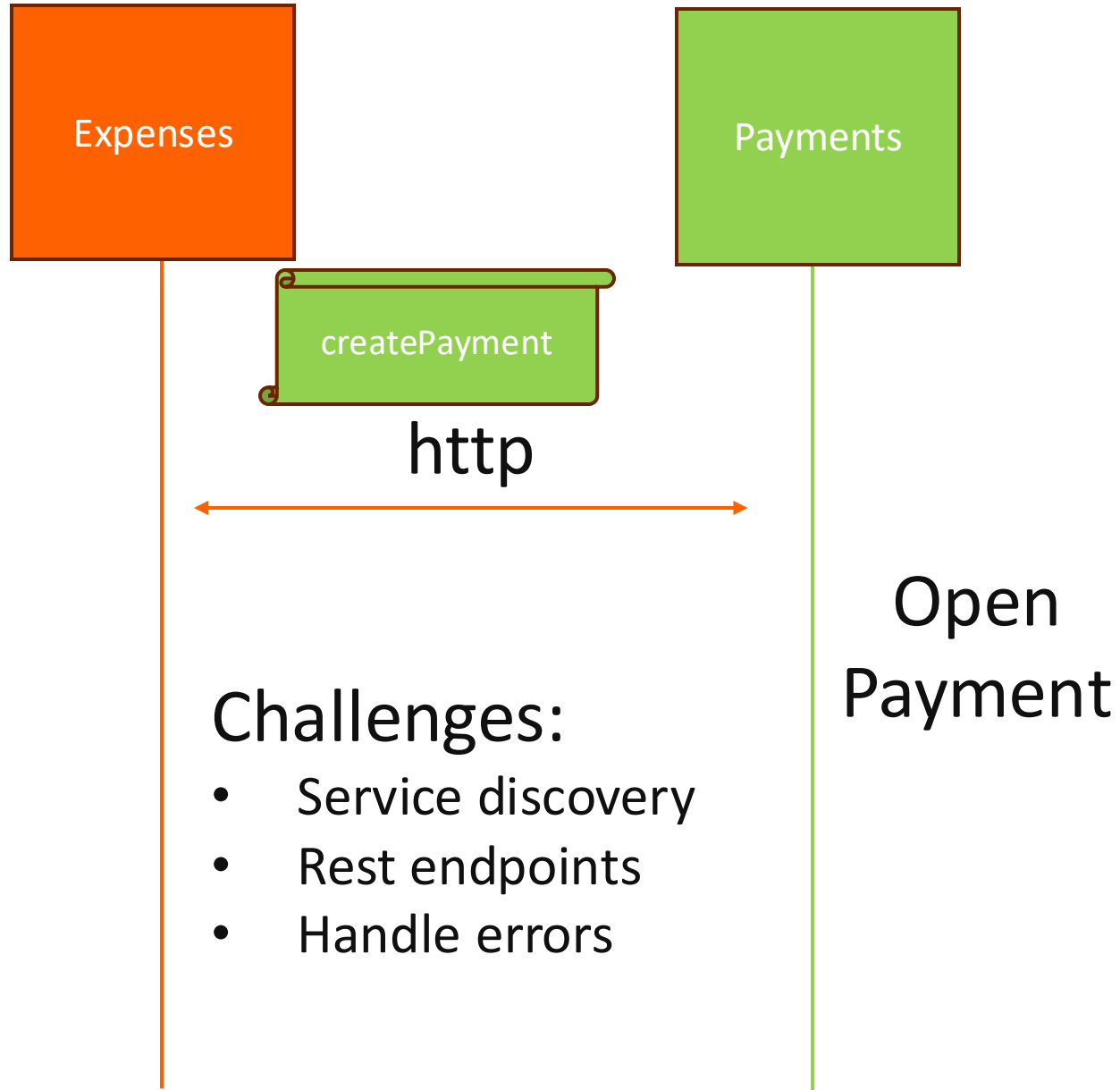
Cost splitting feature - Checkpoints



Demo



Interaction between components



Events

Payment
Opened

Kakfa

Expenses

Expenses

```
public Share claimShare(ExpenseIdentifier expenseId,  
    .filter( Expense e -> {  
        return e.shareCode().isPresent()  
        && e.shareCode().get().value().equals(req.getShareCode());  
    }) Optional<Expense>  
    .map( Expense e -> {  
        var updated :Expense = e.claim(req.getTargetAmount(), currentUserId);  
        Expense saved = repository.update(updated);  
        return saved.getShareNotPaidedByUser(currentUserId)  
            .map( Share s -> {  
                if (s.shareId() != null) {  
                    kafkaPublisher.send(new PaymentOpened(  
                        new InternalReferenceIdentifier(s.id().id())  
                        s.sender(),  
                        s.receiver(),  
                        s.shareAmount()))  
                }  
            })  
    })  
}
```


Events

Amount
Claimed

Kakfa

```
public Share claimShare(ExpenseIdentifier expenseId,  
    .filter( Expense e -> {  
    }) Optional<Expense>  
    .map( Expense e -> {  
        var updated :Expense = e.claim(req.getTargetAmount(), currentUserId);  
        Expense saved = repository.update(updated);  
        return saved.getShareNotPayedByUser(currentUserId)  
        .map( Share s -> {  
            if (s.shouldAllowNewPayments()) {  
                kafkaPublisher.send(new AmountClaimed(  
                    s.id().id(),  
                    s.sender().id(),  
                    s.receiver().id(),  
                    s.shareAmount().targetAmount()))
```

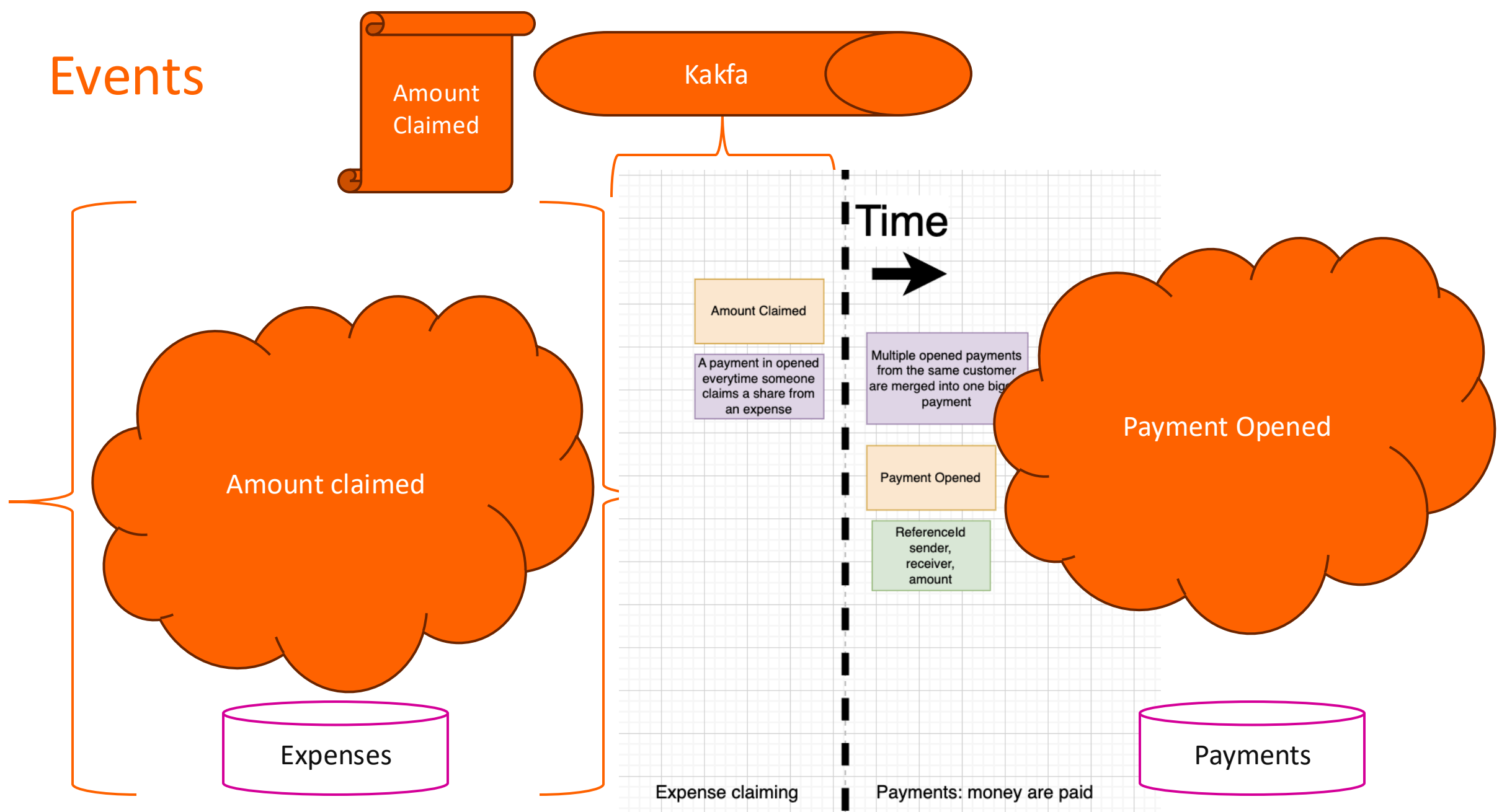
Events

Amount
Claimed

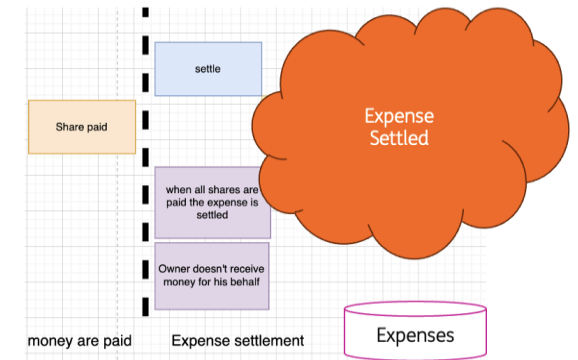
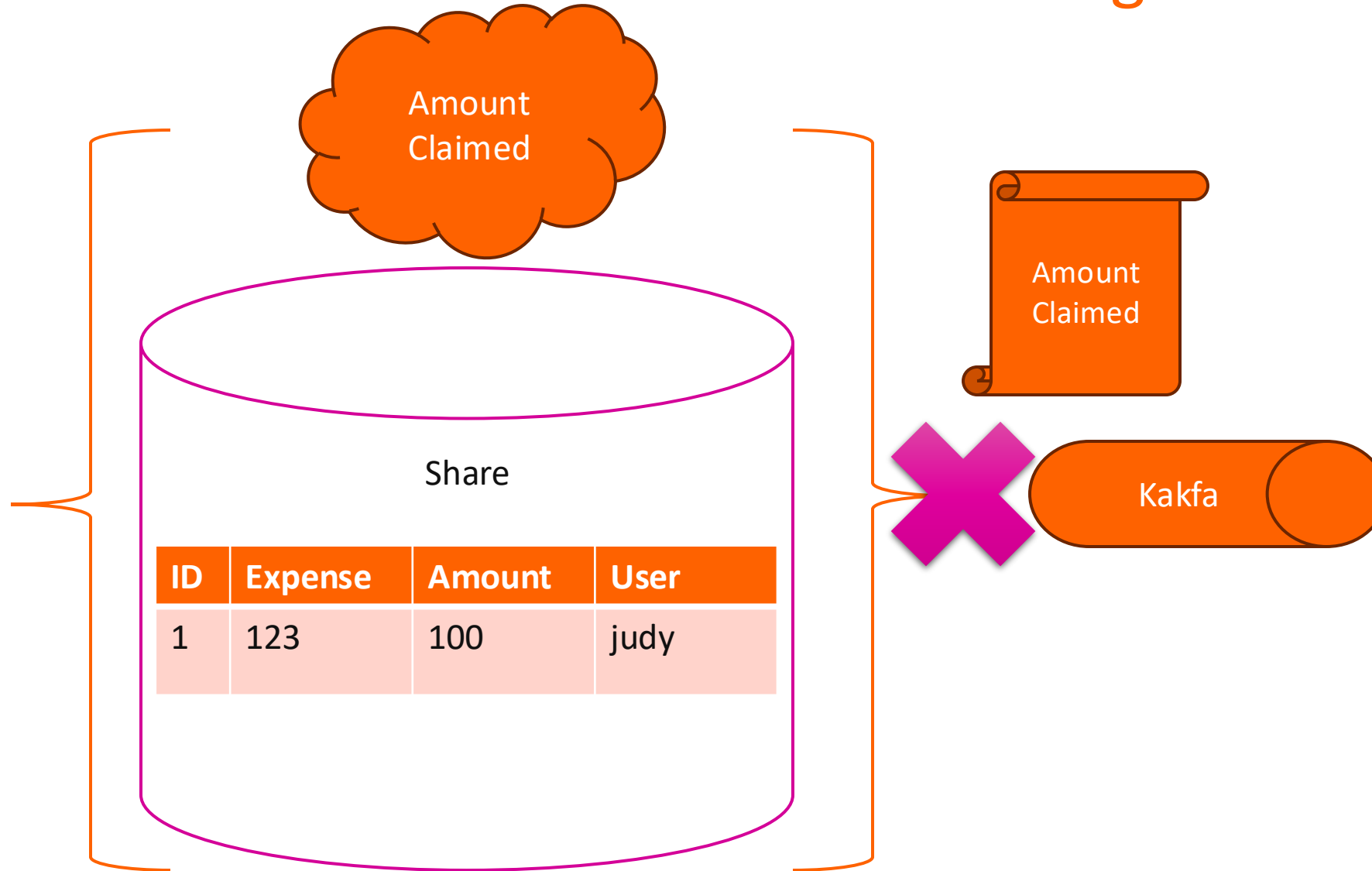
Kafka

```
class ExpenseManagement {  
    public Share claimShare(ExpenseIdentifier expenseId,  
        .filter( Expense e -> {  
            return true,  
        }  
        return e.shareCode().isPresent()  
        && e.shareCode().get().value().equals(req.getShareCode());  
    }) Optional<Expense>  
    .map( Expense e -> {  
        var updated :Expense = e.claim(req.getTargetAmount(), currentUserId);  
        Expense saved = repository.update(updated);  
        saved.getDomainEvents()  
            .forEach(kafkaPublisher::send);  
        return saved.getShareNotPaidByUser(currentUserId)  
            .orElseThrow();  
    }) Optional<Share>  
    .orElseThrow();  
}
```

Events



Reliable event communication using Kafka

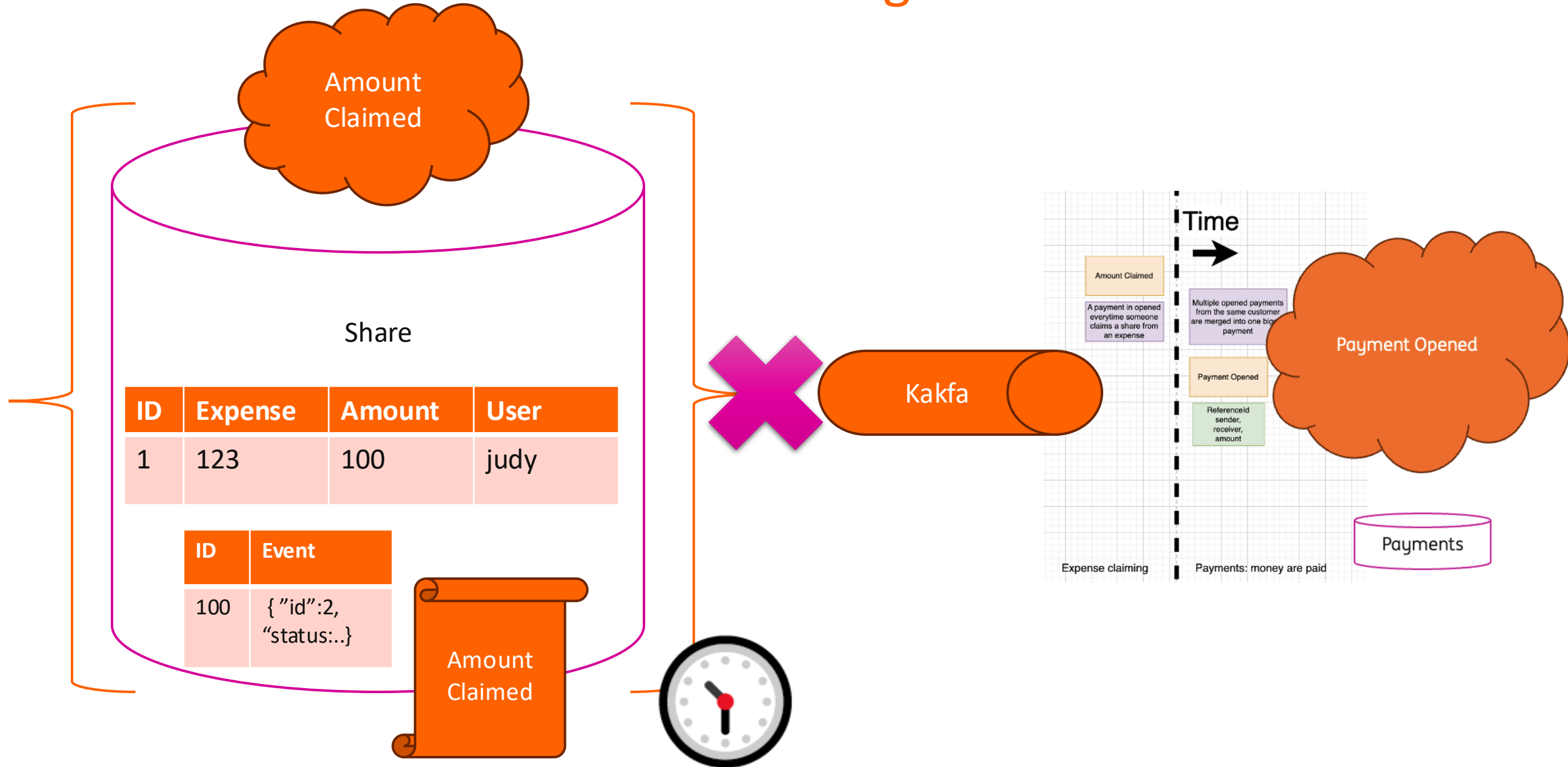


Demo

Unhappy flow



Reliable event communication using Kafka



Demo



Things to consider

Retry policy

Events table cleanup

No side effects if event is consumed multiple times

Refactoring is harder

Complex flows should use event orchestration



Conclusions

- Strategic design is an iterative process
- Impact mapping – aligns business objectives
- Event storming – evolves understanding of the domain model
- Decouple components with events
- Event based communication needs to be async
- Reliable event communication with transactional outbox pattern



Thank you!



github.com/mihaita-tinta/split-mate-poc



medium.com/@mihaita-tinta





do your thing

Follow us @ ING Hubs Romania



inghubsromania.com



LinkedIn.com/company/ing-hubs-romania



Medium.com/ing-tech-romania



Instagram.com/inghubsromania



Facebook.com/inghubsromania



Youtube.com/@INGHubsRomania